Assignment No: 4

Title: Design a responsive webpage using Flexbox, CSS Grid, and media queries.

Aim: To design a fluid and adaptable webpage that automatically adjusts, repositions and optimizes its content across different device screen sizes, from mobile phones to large desktop screens, using CSS Flexbox, CSS Grid and Media Queries.

Objectives:

1. To construct two-dimensional grid layouts using CSS Grid.
2. To align and distribute webpage components using Flexbox.
3. To apply responsive breakpoints using Media Queries.
4. To understand and implement a mobile-first design workflow.
5. To create flexible and adaptable webpage layouts.
6. To develop a webpage that works across mobile, tablet and desktop screen sizes.

Hardware Used:

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

Software Used:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS

Theory:

1. Introduction to Responsive Web Design

Responsive Web Design (RWD) is an approach in which a webpage automatically adjusts its layout, content and visual elements according to the screen size and device being used. CSS creates responsive layouts through flexible dimensions, Flexbox, CSS Grid and Media Queries.

2. Three Core Principles of Responsive Web Design

7. Fluid grids - use relative units such as %, fr, rem and em instead of fixed widths, so elements resize with the available space.
8. Flexible media - images and videos must fit their container (img { max-width: 100%; height: auto; }).
9. Media queries - apply CSS rules conditionally based on viewport width.

3. CSS Flexbox

Flexbox is a one-dimensional layout system used to arrange elements in a row or a column. It is useful for aligning items, distributing space, arranging cards and building responsive headers and footers.

Property	Description
display: flex	Enables Flexbox layout
flex-direction	Defines row or column direction
justify-content	Aligns items along the main axis
align-items	Aligns items along the cross axis
gap	Adds spacing between items
flex-wrap	Allows items to wrap into multiple lines

4. CSS Grid

CSS Grid is a two-dimensional layout system used to arrange elements into rows and columns. It is useful for page layouts, card grids and main-content plus sidebar layouts.

Property	Description
display: grid	Enables Grid layout
grid-template-columns	Defines column sizes
grid-template-rows	Defines row sizes
gap	Adds spacing between grid items
grid-column / grid-row	Defines the position or span of an item

5. Media Queries and Mobile-First Design

Media Queries apply different CSS styles at different screen sizes. In the mobile-first approach the default rules target small screens, and media queries progressively add rules for tablets and desktops.

Example:

```
/* Mobile-first default */
.card-grid { display: flex; flex-direction: column; }

/* Tablet and larger */
@media (min-width: 768px) {
  .card-grid { flex-direction: row; }
}

/* Desktop */
@media (min-width: 1024px) {
  .site-wrapper { flex-direction: row; }
}
```

6. Utility-First Responsive CSS (Tailwind)

A utility framework such as Tailwind CSS expresses the same ideas with prefixed class names. The class grid-cols-1 sm:grid-cols-2 compiles to a grid rule plus an @media (min-width: 640px) override, and hidden lg:flex compiles to display:none plus an @media (min-width: 1024px) rule setting display:flex. The generated stylesheet is ordinary Flexbox, Grid and media-query CSS.

Tailwind prefix	Compiled media query
sm:	@media (min-width: 640px)
md:	@media (min-width: 768px)

lg:	@media (min-width: 1024px)
xl:	@media (min-width: 1280px)

Course Project Implementation:

Course-Project Mapping: Available in the course project. Submit src/components/ResultCard.jsx (CSS Grid with a responsive breakpoint), src/components/Navbar.jsx (Flexbox that switches from column to row) and src/components/PublicView.jsx (a 12-column responsive layout). Because these express the breakpoints through Tailwind prefixes rather than hand-written @media blocks, an equivalent plain-CSS file is also supplied so that the raw media-query syntax is demonstrated explicitly.

Technique	File	Evidence
CSS Grid + breakpoint	src/components/ResultCard.jsx	grid grid-cols-1 sm:grid-cols-2 gap-6
Flexbox direction switch	src/components/Navbar.jsx	flex flex-col lg:flex-row justify-between items-center gap-4
Responsive 12-col layout	src/components/PublicView.jsx	grid lg:grid-cols-12 with lg:col-span-5 / lg:col-span-7
Show / hide by width	src/components/Navbar.jsx	hidden lg:flex and hidden md:block

Program:

Project file - src/components/ResultCard.jsx (CSS Grid, responsive)

```
export default function ResultCard({ result, theme }) {
  if (!result) return null;

  return (
    <div className="animate-[fadeIn_0.5s_ease-out]">
      <h3 className="text-sm font-semibold uppercase mb-4 text-center lg:text-left">
        Analysis Results
      </h3>

      {/* 1 column on mobile, 2 columns from 640px upward */}
      <div className="grid grid-cols-1 sm:grid-cols-2 gap-6 p-6 md:p-8 rounded-xl
border"
           style={{ backgroundColor: theme.headerBg, borderColor: theme.border }}>
        <div className="text-center lg:text-left">
          <div className="text-xs uppercase mb-2">Detected Script</div>
          <div className="text-2xl">{result.script || result.inferredScript}</div>
        </div>
        <div className="text-center lg:text-left">
          <div className="text-xs uppercase mb-2">Historical Era</div>
          <div className="text-2xl md:text-3xl">{result.era || "Unknown"}</div>
        </div>
      </div>
    </div>
  );
}
```

Project file - src/components/Navbar.jsx (Flexbox, responsive)

```
<nav className="sticky top-0 z-50 backdrop-blur-lg border-b shadow-2xl"
     style={{ backgroundColor: theme.headerBg, borderColor: theme.border }}>
  <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">

    {/* stacked column on phones, single row from 1024px upward */}
    <div className="flex flex-col lg:flex-row justify-between items-center py-3 gap-
4">

      <div className="flex items-center gap-12">
        <img src={logo} alt="LipiSutra" className="h-40 w-auto object-contain" />

        {/* page links are hidden on small screens, flex row on large */}
        <div className="hidden lg:flex items-center gap-6 border-l pl-8">
          <button onClick={() => onPageChange("home")}>AI Scanner</button>
        </div>
      </div>

      <div className="flex items-center rounded-full border p-1">
        <span className="text-xs uppercase px-4 hidden md:block">Role</span>
        <div className="flex">
          {roles.map(r => (
            <button key={r.id} onClick={() => onViewChange(r.id)}
                    className="px-5 py-2 text-xs rounded-full border">
              {r.label}
            </button>
          ))}
        </div>
      </div>

    </div>
  </div>
</nav>
```

Supplementary plain-CSS demonstration - assignment04_responsive.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>LipiSutra - Responsive Layout</title>
  <link rel="stylesheet" href="assignment04_responsive.css">
</head>
<body>
  <header class="site-header">
    <h1>LipiSutra</h1>
    <nav class="main-nav">
      <a href="#">Scanner</a><a href="#">Archives</a><a href="#">About</a>
    </nav>
  </header>

  <div class="site-wrapper">
    <main>
      <h2>Recently Decoded Manuscripts</h2>
      <div class="card-grid">
        <article class="card"><h3>Modi Land Grant</h3><p>1712 CE &middot; 92%
confidence</p></article>
        <article class="card"><h3>Devanagari Verse</h3><p>1588 CE &middot; 87%
confidence</p></article>
        <article class="card"><h3>Brahmi Inscription</h3><p>230 BCE &middot; 71%
confidence</p></article>
        <article class="card"><h3>Sharada Folio</h3><p>1104 CE &middot; 80%
confidence</p></article>
      </div>
    </main>
    <aside class="sidebar">
      <h3>Collection Stats</h3>
```

```
      <p>412 documents &middot; 318 verified</p>
    </aside>
  </div>
</body>
</html>
```

Supplementary plain-CSS demonstration - assignment04_responsive.css

```
/* ---------- MOBILE-FIRST DEFAULTS ---------- */
* { box-sizing: border-box; }
body { margin: 0; font-family: Georgia, serif; background: #f5ede0; color: #1a0800; }
img  { max-width: 100%; height: auto; }          /* flexible media */

/* FLEXBOX : header stacks vertically on a phone */
.site-header {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 12px;
  padding: 16px;
  background: #2a1800;
  color: #D4A017;
}
.main-nav { display: flex; flex-wrap: wrap; justify-content: center; gap: 18px; }
.main-nav a { color: #f5e6c8; text-decoration: none; }

/* CSS GRID : one column of cards on a phone */
.card-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}
.card { background: #fff; border: 1px solid #8B4513; border-radius: 10px; padding:
16px; }

/* FLEXBOX : main and sidebar stacked on a phone */
.site-wrapper { display: flex; flex-direction: column; gap: 20px; padding: 20px; }
.sidebar { background: #ede0cc; border-radius: 10px; padding: 16px; }

/* ---------- TABLET : 768px and above ---------- */
@media (min-width: 768px) {
  .site-header  { flex-direction: row; justify-content: space-between; padding: 16px
32px; }
  .card-grid    { grid-template-columns: repeat(2, 1fr); }   /* 2 columns */
}

/* ---------- DESKTOP : 1024px and above ---------- */
@media (min-width: 1024px) {
  .site-wrapper { flex-direction: row; padding: 32px; }
  .site-wrapper main { flex: 7; }                            /* 7/12 of the row */
  .sidebar           { flex: 3; align-self: flex-start; }    /* 3/12 of the row */
  .card-grid         { grid-template-columns: repeat(4, 1fr); }
}

/* ---------- LARGE DESKTOP : 1280px and above ---------- */
@media (min-width: 1280px) {
  .site-wrapper { max-width: 1240px; margin: 0 auto; }
}
```

Output:

- Below 768px: the header is stacked, the cards form a single column and the sidebar sits under the main content.

- At 768px the card grid reflows to two columns and the header becomes a single row with the navigation pushed to the right.
- At 1024px the wrapper becomes a row, giving a 7:3 main-and-sidebar split, and the cards reflow to four columns.
- The same behaviour is observable in the running LipiSutra application: resizing the window below 1024px collapses the navbar to a column and hides the AI Scanner link, and below 640px the ResultCard script/era pair stacks vertically.

Conclusion:

In this assignment, we designed a responsive webpage using Flexbox for one-dimensional alignment, CSS Grid for two-dimensional card layouts and media queries for breakpoints, following a mobile-first workflow. We also showed that the LipiSutra project implements exactly these techniques through Tailwind responsive prefixes, which compile to the same @media rules written by hand in the supplementary stylesheet.

References:

10. Mozilla Developer Network (MDN) Web Docs - CSS Basics, https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics
11. W3Schools - CSS Tutorial, https://www.w3schools.com/css/
12. CSS Specifications - W3C, https://www.w3.org/Style/CSS/
13. MDN Web Docs - CSS Flexible Box Layout, https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout
14. MDN Web Docs - CSS Grid Layout, https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout